SOP Versioning and Unreal Engine Build Automation

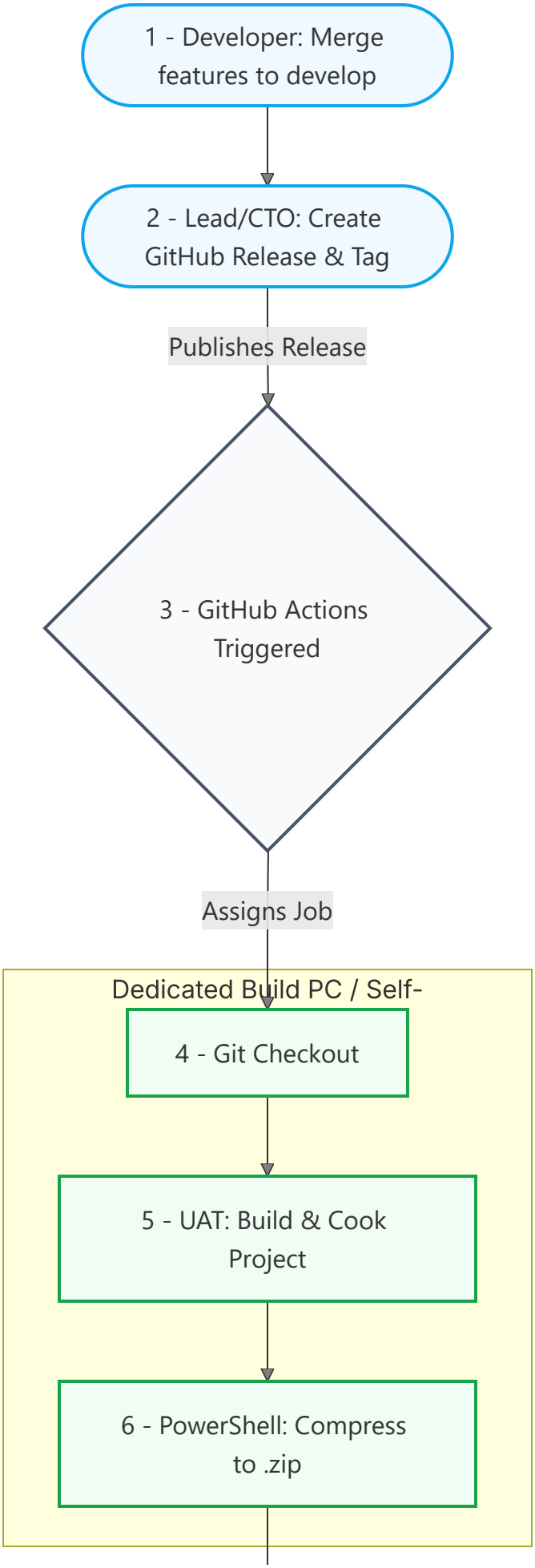
Author: Aryo, CTO
Date: May 5, 2026
Status: Professional Standard Operating Procedure

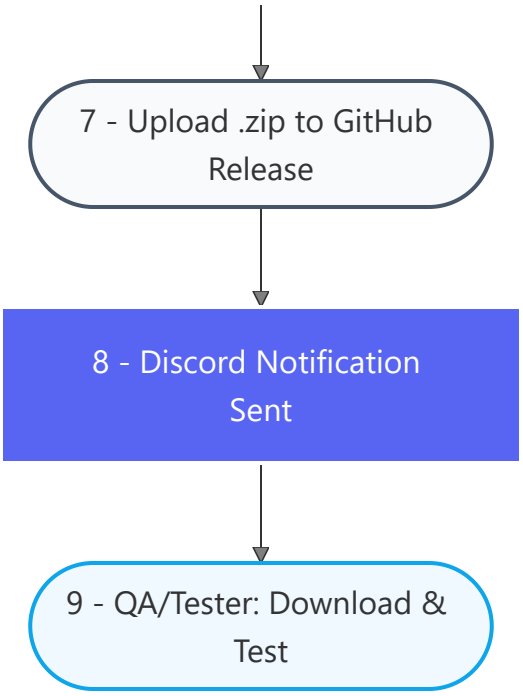
📄 Introduction

This document establishes the standard operating procedure (SOP) for versioning and automated deployment within our Unreal Engine projects. By leveraging **GitHub Actions** and **Self-Hosted Runners**, we eliminate manual packaging time, ensure consistent build environments, and provide real-time transparency to Project Managers and Stakeholders.

🔄 Workflow Architecture

This flowchart defines the end-to-end lifecycle of a build, from the initial code merge to the final notification sent to the team.


1 - Developer: Merge features to develop
2 - Lead/CTO: Create GitHub Release & Tag
Publishes Release
3 - GitHub Actions Triggered
Assigns Job
Dedicated Build PC / Self-
4 - Git Checkout
5 - UAT: Build & Cook Project
6 - PowerShell: Compress to .zip



PHASE 1: INFRASTRUCTURE SETUP

This setup is performed once per project or per build machine.

1. Build Machine Prerequisites

- **Hardware:** Dedicated Windows PC with high-speed SSD and recomended 32GB RAM. Minimum 16GB
- **Software:**
  - Unreal Engine (Target Version).
  - Visual Studio (matching UE version) with **"Game development with C++"**.
  - Git for Windows.
- **Authentication:** Set up SSH keys or Git Credential Manager to allow the machine to pull code without manual login prompts.

2. GitHub Runner Registration

1. Navigate to **Settings > Actions > Runners** in the GitHub Repository.
2. Select **New self-hosted runner** and choose **Windows**.
3. Follow the PowerShell instructions provided to download and configure the agent.
4. **Pro-Tip:** Run `./config.cmd --help` to see how to install the runner as a **Windows Service** so it starts automatically on reboot.

3. Repository Hygiene (`.gitignore`)

Ensure your `.gitignore` is optimized for Unreal to prevent unnecessary files (like `Intermediate` or `Saved`) from being uploaded.

PHASE 2: VERSIONING STRATEGY

We use **Semantic Versioning (SemVer)** to communicate the nature of changes to the rest of the team.

1. Version Numbering Rules

Component	Name	Description	When to Increment
v1.x.x	**Major**	Significant changes.	Breaking API changes or major engine version upgrades.
x.**2**.x	**Minor**	New Features.	When adding functionality that doesn't break existing builds.
x.x.**3**	**Patch**	Bug Fixes.	Small fixes, hotfixes, or performance optimizations.

2. Pre-Release Tags & Selection Criteria

Tag	Status	Meaning	Target Audience
`-alpha`	**Experimental**	New features, high probability of bugs.	Internal Dev Team
`-beta`	**Feature Complete**	All features are in; testing stability.	Internal/External QA
`-rc`	**Release Candidate**	Believed to be stable; pending final sign-off.	Stakeholders / PM
(None)	**Stable/Final**	Production-ready version.	Clients / End Users

PHASE 3: CI/CD & NOTIFICATION SETUP

1. Discord Integration

1. **Create Webhook:** Discord Channel Settings > Integrations > Webhooks > New Webhook.
2. **Add Secret:** In GitHub, go to **Settings > Secrets and variables > Actions > New repository secret**.
   - **Name:** `DISCORD_WEBHOOK`
   - **Value:** `[Paste your Discord Webhook URL]`

2. Automation Workflow (`unreal_build.yml`)

Create this file at `.github/workflows/unreal_build.yml`. This script is **generic** and will auto-detect the `.uproject` file in your repository.

```yaml
name: Unreal Engine Build Automation

on:
  release:
    types: [published]

jobs:
  build_unreal:
    runs-on: self-hosted

    steps:
      - name: Checkout Code
        uses: actions/checkout@v4

      - name: Setup Environment Variables
        run: |
          $TAG_NAME = "${{ github.event.release.tag_name }}"
          $REPO_NAME = "${{ github.event.repository.name }}"
          echo "ZIP_NAME=${REPO_NAME}_${TAG_NAME}.zip" >> $env:GITHUB_ENV
          echo "BUILD_DIR=${{ github.workspace }}\Build_Output" >> $env:GITHUB_ENV

      - name: Build & Cook (UAT)
        run: |
          # UPDATE THIS PATH TO YOUR ACTUAL UE INSTALLATION
          $UAT_Path = "C:\Program Files\Epic
Games\UE_5.3\Engine\Build\BatchFiles\RunUAT.bat"

          $ProjectFile = Get-ChildItem -Path "${{ github.workspace }}" -Filter
*.uproject | Select-Object -First 1
          if (-not $ProjectFile) { throw "No .uproject file found!" }

          & $UAT_Path BuildCookRun -project="$($ProjectFile.FullName)" -noP4 -
platform=Win64 -clientconfig=Development -cook -allmaps -build -stage -pak -archive -
archivedirectory="${{ env.BUILD_DIR }}"

      - name: Compress Build
        run: |
          Compress-Archive -Path "${{ env.BUILD_DIR }}\Windows\*" -DestinationPath
"${{ env.ZIP_NAME }}" -Force

      - name: Upload to GitHub Release
        uses: svenstaro/upload-release-action@v2
        with:
          repo_token: ${{ secrets.GITHUB_TOKEN }}
          file: ${{ env.ZIP_NAME }}
          asset_name: ${{ env.ZIP_NAME }}
          tag: ${{ github.ref }}
          overwrite: true
```

```
      - name: Notify Discord
        if: always()
        uses: appleboy/discord-action@master
        with:
          webhook_id: ${{ secrets.DISCORD_WEBHOOK }}
          message: |
            **🚀 Build Complete: ${{ github.event.repository.name }}**
            **Version:** `${{ github.event.release.tag_name }}`
            **Status:** ${{ job.status == 'success' && '✅ Success' || '❌ Failed' }}
            **Download:** [GitHub Release Page](${{ github.event.release.html_url }})
            ---
            *Triggered by: ${{ github.actor }}*
```

PHASE 4: BEST PRACTICES & VERIFICATION

1. **Local Validation:** Before tagging a release, ensure the project compiles locally on the developer's machine.
2. **Versioning Discipline:** Never skip a version number. If `v0.1.0-rc.1` fails, the next fix is `v0.1.0-rc.2`.
3. **Engine Path Check:** Always verify the `$UAT_Path` in your `.yml` file if you update the Unreal Engine version.
4. **Build PC Maintenance:** Clean the `Build_Output` folder periodically to save disk space.